

Layer-Adaptive XMSSMT Parameterization: A Multi-Objective Study of WOTS Placement, Tree-Height Allocation, and Serialized State

Sohaib Rana¹, Mohd Najwadi Yusoff^{1,2*}, Je Sen Teh^{3,4*},
Adnan Anwar³

¹School of Computer Sciences, Universiti Sains Malaysia, Gelugor, Penang, Malaysia.

²Cybersecurity Research Centre (CYRES), Universiti Sains Malaysia, Gelugor, Penang, Malaysia.

³School of Information Technology, Deakin University, Waurin Ponds, Victoria, Australia.

⁴Deakin Cyber, Deakin University, Waurin Ponds, Victoria, Australia.

*Corresponding author(s). E-mail(s): najwadi@usm.my;
j.teh@deakin.edu.au;

Contributing authors: sohaibrana2023@student.usm.my;
adnan.anwar@deakin.edu.au;

Abstract

XMSSMT implementations commonly use uniform per-layer parameters, although lower and upper layers perform asymmetric computational and state-management roles. This study evaluates ordered layer-specific tree-height and WOTS parameterization in a two-layer XMSSMT FAST/BDS implementation with total height $H = 20$. Five height allocations, $(6, 14)$, $(8, 12)$, $(10, 10)$, $(12, 8)$, and $(14, 6)$, are combined with four ordered WOTS placements, $(16, 16)$, $(16, 4)$, $(4, 16)$, and $(4, 4)$, yielding 20 experimental configurations. A unified Raspberry Pi 5 protocol measures one complete lower-layer traversal cycle plus rollover for every configuration, producing 87,316 signing observations with matched verification, hardware-cycle, and logical-work measurements. Lower-layer WOTS placement dominates routine signing cost: across height allocations, $(4, 16)$ reduces mean signing latency substantially relative to $(16, 4)$ despite both producing 7075-byte signatures. Serialized-state size follows the exact relation $|\mathbf{SK}| = 2654 + 120h_0 + W_1$, while signature size follows

$|\sigma| = 675 + W_0 + W_1$; both models reproduce all 20 measured configurations exactly. Rollover behavior is composition-dependent rather than a fixed positive surcharge, with **(4, 16)** showing positive rollover increases and **(16, 4)** negative changes across all tested height allocations. An aggregate workload model explains measured CPU cycles with $R^2 = 0.9880$. Under the primary four-objective analysis of mean signing latency, P95 signing latency, signature size, and serialized state, three configurations form the strict Pareto front; two remain robust under 2%, 5%, and 10% tolerance. The results quantify ordered layer effects in a concrete implementation and do not claim that heterogeneous XMSSMT parameterization itself is new.

Keywords: XMSSMT, XMSS, hash-based signatures, post-quantum cryptography, WOTS+, Winternitz parameter, BDS traversal, heterogeneous parameterization, Raspberry Pi 5, multi-objective optimization

1 Introduction

Hash-based signatures are among the most mature approaches to post-quantum digital signatures because their security can be based on well-studied properties of cryptographic hash functions rather than on number-theoretic assumptions. XMSS was introduced as a forward-secure stateful hash-based signature scheme and later extended through the multi-tree construction XMSSMT [1, 2]. XMSS and XMSSMT use WOTS+ as their one-time signature component [2, 3].

The multi-tree construction reduces the cost associated with very large single Merkle trees by dividing the total tree height across multiple layers. This introduces additional design parameters, including the number of layers, subtree heights, Winternitz parameters, and traversal parameters. These choices jointly affect signing cost, verification cost, signature size, traversal state, and the computational behavior at subtree boundaries.

Stateful hash-based signatures are standardized for restricted deployment scenarios. RFC 8391 specifies XMSS and XMSSMT [2], while RFC 8554 specifies LMS and its hierarchical extension HSS [4]. NIST SP 800-208 profiles XMSS, XMSSMT, LMS, and HSS for stateful hash-based signature applications [5]. NIST emphasizes that such schemes require careful state management because reuse of one-time signing state can invalidate their security guarantees.

The XMSSMT parameter-selection problem has been studied previously. In particular, Hülsing, Rausch, and Buchmann considered optimization of layer-dependent XMSSMT parameters, including subtree heights, traversal parameters, and Winternitz parameters [6]. Consequently, the use of heterogeneous or layer-specific XMSSMT parameters is not itself a new construction.

A remaining implementation question is how the *ordered placement* of these parameters manifests in a concrete FAST/BDS implementation when configurations are compared under an identical measurement protocol. Two parameterizations can have the same total tree height and even the same transmitted signature size while assigning computational work differently between the frequently executed lower layer and

the less frequently refreshed upper layer. In addition, BDS traversal makes signing cost index-dependent, so comparisons based only on short early-index signing windows can obscure traversal bursts and rollover behavior.

This work therefore studies a controlled two-layer XMSSMT design space with fixed total height $H = 20$ and layer-specific height and WOTS parameters. Five ordered height allocations,

$$(6, 14), (8, 12), (10, 10), (12, 8), (14, 6),$$

are combined with four ordered WOTS placements,

$$(16, 16), (16, 4), (4, 16), (4, 4),$$

to produce 20 experimental configurations. These configurations are used as an implementation study and are not claimed to be standardized XMSSMT parameter sets.

The evaluation differs from a parameter-only comparison in five ways. First, all configurations are measured using one matched full-cycle protocol covering a complete lower-layer traversal cycle and its rollover boundary. Second, the implementation records both wall-clock latency and hardware CPU cycles at operation level. Third, algorithmic work counters are collected to relate observed performance to logical XMSSMT operations. Fourth, signature and serialized-state sizes are derived analytically and checked against every measured configuration. Fifth, deployment trade-offs are evaluated using strict and tolerance-based multi-objective Pareto analysis.

The main contributions are therefore as follows:

- a FAST/BDS implementation of ordered layer-specific XMSSMT height and WOTS parameterization under a single matched full-cycle benchmark protocol;
- an operation-level characterization of lower-layer placement, traversal burstiness, and rollover behavior using latency, hardware cycles, and logical-work counters;
- exact analytical models for signature size and serialized implementation state, validated with zero-byte residuals across the complete 20-configuration matrix;
- an empirical aggregate workload model relating logical hash work and tree-node generation to measured processor cycles; and
- a multi-objective analysis of signing latency, tail latency, signature size, serialized state, and verification sensitivity, including robustness under symmetric ϵ -dominance.

The contribution is thus not a new XMSSMT cryptographic primitive or the first proposal of heterogeneous XMSSMT parameters. Rather, it is an implementation-level quantification and validation of ordered layer-placement effects, full-cycle traversal behavior, rollover composition, serialized-state structure, and deployment trade-offs on a Raspberry Pi 5 platform.

2 Related Work

2.1 XMSS, WOTS+, and XMSSMT

XMSS was introduced by Buchmann, Dahmen, and Hülsing as a practical forward-secure hash-based signature scheme [1]. WOTS+ subsequently strengthened the Winternitz one-time signature construction while reducing signature length relative to earlier variants [3]. RFC 8391 later specified interoperable WOTS+, XMSS, and XMSSMT constructions [2].

XMSSMT organizes multiple XMSS trees hierarchically. A lower-layer tree signs messages, while its root is authenticated by a WOTS+ signature in the next layer. This organization changes how computation is distributed across layers: lower-layer operations occur frequently, whereas upper-layer authentication material can be reused until a lower subtree is exhausted. The placement of layer-specific parameters can therefore affect execution cost even when aggregate parameters such as total tree height or total WOTS signature bytes remain unchanged.

2.2 XMSSMT Parameter Optimization

Hülsing, Rausch, and Buchmann explicitly studied optimal parameter selection for XMSSMT [6]. Their formulation allows parameters to vary across layers and considers the interaction between subtree height, traversal parameter, and Winternitz parameter. That work establishes important prior art for heterogeneous XMSSMT parameterization and prevents interpreting layer-specific (h_i, k_i, w_i) choices as a novel concept in themselves.

The focus of the present work is different. Rather than introducing the idea of heterogeneous parameters, we experimentally isolate the effect of their *ordered placement* in a concrete FAST/BDS implementation. The study additionally measures an entire lower-layer traversal cycle, including rollover, and records operation-level hardware cycles and algorithmic work counters. This provides an empirical view of effects that are structural in the scheme but can be obscured by limited signing windows or by aggregate timing alone.

2.3 Standardized Stateful Hash-Based Signatures

RFC 8391 specifies XMSS and XMSSMT and defines interoperable parameter sets for those schemes [2]. The heterogeneous configurations used in this work are experimental implementation variants and are not assigned or represented as standardized RFC parameter identifiers.

LMS and HSS provide a second standardized stateful hash-based signature family. RFC 8554 specifies LMS and the hierarchical HSS construction [4]. HSS is relevant to the present discussion because hierarchical hash-based signatures generally expose design trade-offs between lower-level signing work, upper-level authentication, key lifetime, and state management.

NIST SP 800-208 profiles both families: XMSS/XMSSMT and LMS/HSS [5]. The recommendation approves selected parameter sets rather than arbitrary experimental combinations and emphasizes the operational importance of maintaining state

correctly. Accordingly, the parameter sweep in this study should be viewed as an implementation and design-space experiment rather than as a proposal to redefine the standardized profiles.

2.4 Positioning of the Present Study

Prior work establishes the underlying XMSS construction, WOTS+, multi-tree organization, and optimization of XMSSMT parameters. The present study addresses a narrower empirical question: how do ordered layer-specific height and WOTS choices interact with FAST/BDS traversal, serialized implementation state, full-cycle signing cost, and rollover behavior when all configurations are measured using an identical operation-level protocol?

This distinction determines the novelty claim used throughout the paper. The contribution lies in matched implementation evidence and analytical validation, not in claiming the first use of heterogeneous XMSSMT parameters. In particular, the comparison between $(w_0, w_1) = (4, 16)$ and $(16, 4)$ demonstrates why ordered placement deserves explicit measurement: both configurations have the same transmitted signature size, yet assign work differently between the continuously active lower layer and the periodically refreshed upper layer.

3 Methods

3.1 Experimental Design

We evaluate a two-layer XMSSMT construction with total Merkle-tree height $H = 20$ and $d = 2$ layers. Unlike standardized XMSSMT parameter sets, which use a common subtree height and Winternitz parameter across layers, the configurations considered here are experimental heterogeneous parameterizations intended to isolate the effects of ordered layer placement. The layer heights satisfy $h_0 + h_1 = 20$, with

$$(h_0, h_1) \in \{(6, 14), (8, 12), (10, 10), (12, 8), (14, 6)\},$$

and the layer-specific WOTS parameters are

$$(w_0, w_1) \in \{(16, 16), (16, 4), (4, 16), (4, 4)\}.$$

Their Cartesian product yields 20 configurations. No standardized OIDs are claimed for these heterogeneous configurations.

3.2 FAST/BDS Implementation

Measurements use the FAST XMSSMT implementation with BDS traversal. For all evaluated configurations, the BDS parameter is $k = 0$. For $d = 2$, the serialized traversal state contains the current lower-layer state, the current upper-layer state, and a next lower-layer state. The implementation additionally caches the upper-layer WOTS signature used between lower-tree rollover events. Consequently, serialized state size

includes both traversal data and cached public WOTS-signature material and should not be interpreted as entirely secret memory.

The implementation invokes SHA-256 and SHA-512 through OpenSSL `libcrypto.so.3`. The Raspberry Pi processor exposes the AArch64 `sha2` feature. These observations establish the software and processor capabilities used by the experiment but do not by themselves identify the exact OpenSSL internal assembly path selected at run time.

3.3 Unified Full-Cycle Measurement Protocol

The revised evaluation uses a single matched measurement protocol for all 20 configurations. This replaces earlier comparisons based on different signing windows. For each configuration, ten key-generation operations are measured. Signing is then measured over one complete lower-layer traversal cycle together with the first signature after the rollover boundary. Therefore, a configuration with lower-layer height h_0 contributes

$$2^{h_0} + 1$$

measured signing operations.

Each measured signature is immediately verified, so signing and verification are evaluated over the same index sequence. The resulting signing sample counts are 65, 257, 1025, 4097, and 16385 for $h_0 = 6, 8, 10, 12$, and 14, respectively. Across the complete 20-configuration matrix, the unified protocol produces 87,316 measured signing operations.

Before measurement, three key-generation operations and 100 signing operations are executed on a disposable key as an in-process warm-up. The benchmark records the serialized signature index before and after each signing operation and requires an increment of exactly one. This provides an explicit state-progression check and identifies the lower-layer rollover operation at

$$i = 2^{h_0} - 1.$$

3.4 Platform and Timing

Experiments were executed on a Raspberry Pi 5 Model B Rev. 1.1 with a Broadcom BCM2712 processor using an AArch64 Cortex-A76 CPU. The system ran Debian 13 (trixie) with the measured process pinned to CPU 3 under the `performance` frequency governor. Observed CPU-frequency samples were 2.4 GHz. Boost was disabled, and the platform throttling indicator remained 0x0 throughout the campaign.

Wall-clock latency is measured with `CLOCK_MONOTONIC_RAW`. Hardware CPU cycles are collected for each key-generation, signing, and verification operation using `perf_event_open()` with `PERF_COUNT_HW_CPU_CYCLES`, excluding kernel and hypervisor execution. The measured kernel remained `6.18.34+rpt-rpi-2712` during the campaign.

3.5 Logical Work Instrumentation

In addition to latency and hardware cycles, the benchmark records algorithm-level work counters for each operation: PRF invocations, F -hash invocations, H -hash invocations, message-hash invocations, WOTS chain steps, generated tree nodes, authentication-path nodes, and BDS updates. We define

$$N_{\text{logical}} = N_{\text{PRF}} + N_F + N_H + N_{\text{msg}}.$$

These counters denote logical XMSS/XMSSMT hash-function invocations and must not be interpreted as counts of SHA-256 compression-function calls.

The instrumentation shows that WOTS chain steps and F -hash calls are identical in the present implementation. Strong collinearity also exists among several primitive counters. Therefore, primitive-level regression coefficients are not interpreted as independent operation costs.

3.6 Summary Statistics

Because BDS traversal produces index-dependent and bursty signing costs, signing observations are treated as an ordered traversal sequence rather than as independent and identically distributed samples. For each configuration we report the arithmetic mean, median, sample standard deviation, coefficient of variation, minimum, maximum, and 95th percentile. Percentiles use linear interpolation at position $(n-1)p$.

The complete lower-layer cycle is retained without deleting wall-clock outliers. Hardware-cycle measurements and logical work counters are reported alongside latency to distinguish algorithmic work variation from host-level scheduling or operating-system effects.

3.7 Rollover Analysis

For every configuration, the rollover operation is defined as the signature whose serialized index before signing equals $2^{h_0} - 1$. Its cost is compared with the mean of the immediately preceding and following signing operations. We report

$$\Delta_{\text{roll}} = 100 \frac{T_{\text{roll}} - (T_{\text{pre}} + T_{\text{post}})/2}{(T_{\text{pre}} + T_{\text{post}})/2}.$$

A rollover is not assumed to impose a fixed positive surcharge. At the lower-tree boundary the implementation changes traversal-state composition and generates a fresh upper-layer WOTS signature. The resulting latency change can therefore be positive or negative, depending on the ordered WOTS parameters and the work displaced at the boundary.

3.8 Analytical Artifact-Size Model

For $n = 32$ and $k = 0$, the serialized BDS state for the even subtree heights used in this study reduces to

$$S(h) = 120h + 41 \quad \text{bytes}.$$

For the two-layer FAST topology, the serialized XMSSMT state is

$$|\text{SK}| = 131 + 2S(h_0) + S(h_1) + W_1,$$

where W_1 is the cached upper-layer WOTS-signature size. Since $h_0 + h_1 = 20$,

$$|\text{SK}| = 2654 + 120h_0 + W_1.$$

The WOTS signature sizes are 4256 bytes for $w = 4$ and 2144 bytes for $w = 16$. Hence,

$$|\text{SK}| = \begin{cases} 4798 + 120h_0, & w_1 = 16, \\ 6910 + 120h_0, & w_1 = 4. \end{cases}$$

The XMSSMT signature size is

$$|\sigma| = 675 + W_0 + W_1,$$

which yields 4963, 7075, 7075, and 9187 bytes for $(w_0, w_1) = (16, 16), (16, 4), (4, 16)$, and $(4, 4)$, respectively. The analytical signature and serialized-state formulas reproduce all 20 measured artifact sizes exactly, with zero-byte residuals.

3.9 Aggregate Workload Model

To relate logical algorithmic work to measured processor cost, an aggregate empirical model is fitted over all 87,316 signing observations:

$$C = \alpha + \beta N_{\text{logical}} + \gamma N_{\text{tree}},$$

where C is measured CPU cycles and N_{tree} is the number of generated tree nodes.

For the Raspberry Pi 5 campaign, the fitted model is

$$C = -157535.67 + 1779.238 N_{\text{logical}} + 56185.960 N_{\text{tree}}.$$

The model achieves $R^2 = 0.9880$, a median absolute percentage error of 4.348%, and a 95th-percentile absolute percentage error of 6.757%. These coefficients are platform- and workload-specific empirical calibration parameters and are not interpreted as universal costs of the underlying hash primitives.

3.10 Multi-Objective Analysis

The primary deployment-oriented Pareto analysis minimizes four objectives simultaneously: mean signing latency, P95 signing latency, signature size, and serialized state size. A strict Pareto front is computed first. Robustness is then examined using symmetric ϵ -dominance for $\epsilon \in \{2\%, 5\%, 10\%\}$.

Under this rule, configuration A is considered no worse than configuration B when every objective of A is at most $(1 + \epsilon)$ times the corresponding objective of B , with at least one objective smaller than $(1 - \epsilon)$ times that of B . A secondary sensitivity analysis additionally includes mean verification latency as a fifth objective.

Table 1 Full-cycle signing, verification, rollover, and artifact-size results for the 20 heterogeneous XMSSMT configurations on Raspberry Pi 5. Mean, median, and P95 signing latency are computed over one complete lower-layer traversal cycle plus the first signature after rollover. Rollover Δ is measured relative to the mean of the immediately preceding and following signatures. Serialized state includes traversal state and cached upper-layer WOTS signatures.

h_0	h_1	w_0	w_1	Sign latency (ms)			Cycles (M)	Verify (ms)	Rollover Δ (%)	Sig. (B)	State (B)
				Mean	Median	P95					
6	14	16	16	8.482	7.064	13.266	20.339	2.607	18.33	4963	5518
6	14	16	4	8.346	6.134	13.107	20.018	2.161	-8.45	7075	7630
6	14	4	16	4.623	4.528	7.233	11.087	2.242	68.99	7075	5518
6	14	4	4	4.599	3.659	7.247	11.030	1.608	13.57	9187	7630
8	12	16	16	10.674	12.957	15.469	25.598	2.549	20.44	4963	5758
8	12	16	4	10.675	12.988	15.496	25.601	2.123	-9.33	7075	7870
8	12	4	16	5.966	7.278	8.675	14.308	2.146	71.62	7075	5758
8	12	4	4	5.892	7.189	8.581	14.129	1.707	17.83	9187	7870
10	10	16	16	13.066	15.435	17.913	31.336	2.605	18.64	4963	5998
10	10	16	4	13.030	15.389	17.874	31.249	2.168	-6.57	7075	8110
10	10	4	16	7.240	8.571	9.947	17.364	2.120	71.36	7075	5998
10	10	4	4	7.227	8.555	9.932	17.332	1.674	16.97	9187	8110
12	8	16	16	15.469	17.838	20.299	37.022	2.746	21.43	4963	6238
12	8	16	4	15.430	17.826	20.283	36.996	2.153	-10.94	7075	8350
12	8	4	16	8.532	9.869	11.232	20.460	2.177	64.13	7075	6238
12	8	4	4	8.541	9.882	11.241	20.482	1.649	17.54	9187	8350
14	6	16	16	17.891	20.330	22.774	42.907	2.488	24.04	4963	6478
14	6	16	4	19.682	22.371	25.074	47.201	2.385	-7.35	7075	8590
14	6	4	16	9.907	11.264	12.619	23.759	2.119	74.29	7075	6478
14	6	4	4	10.928	12.425	13.926	26.207	1.834	15.20	9187	8590

Table 2 Robustness of the primary four-objective Pareto set under symmetric ϵ -dominance. The objectives are mean signing latency, P95 signing latency, signature size, and serialized state size.

Configuration	Strict	2%	5%	10%	Mean Sign (ms)		Sig. (B)	State (B)
						P95		
H6-14-W16-16	Yes	Yes	Yes	Yes	8.482	13.266	4963	5518
H6-14-W4-16	Yes	Yes	Yes	Yes	4.623	7.233	7075	5518
H6-14-W4-4	Yes	—	—	—	4.599	7.247	9187	7630

4 Results

4.1 Full-Cycle Signing Cost and Layer Placement

Table 1 summarizes the matched full-cycle measurements for all 20 heterogeneous XMSSMT configurations. Figure 1 shows mean signing latency as a function of the lower-layer height h_0 for each ordered WOTS placement.

Across all four WOTS placements, increasing h_0 generally increases full-cycle signing cost. For $(w_0, w_1) = (16, 16)$, mean signing latency rises from 8.482 ms at $(h_0, h_1) = (6, 14)$ to 17.891 ms at $(14, 6)$. The corresponding mean hardware-cycle count rises from 20.339 to 42.907 million cycles. The same structural trend appears for the $w_0 = 4$ configurations, although at a substantially lower signing cost.

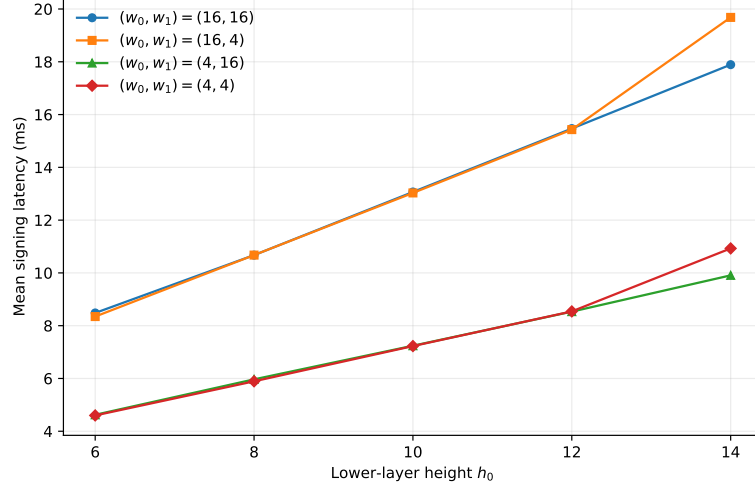


Fig. 1 Mean full-cycle signing latency versus lower-layer height h_0 for the four ordered WOTS placements. Each point is computed from one complete lower-layer traversal cycle plus the first signature after rollover.

The lower-layer WOTS parameter has the dominant effect on ordinary full-cycle signing. With $w_1 = 16$, changing w_0 from 16 to 4 reduces mean signing latency from 8.482 to 4.623 ms at $h_0 = 6$, from 10.674 to 5.966 ms at $h_0 = 8$, from 13.066 to 7.240 ms at $h_0 = 10$, from 15.469 to 8.532 ms at $h_0 = 12$, and from 17.891 to 9.907 ms at $h_0 = 14$. These reductions are approximately 44–46% across the tested height allocations.

In contrast, changing only w_1 generally has a much smaller effect on the full-cycle mean because the upper-layer WOTS signature is cached between lower-tree rollovers rather than regenerated for every signature. This distinction is visible, for example, at $(h_0, h_1) = (10, 10)$: mean signing latency is 13.066 ms for (16, 16) and 13.030 ms for (16, 4), while it is 7.240 ms for (4, 16) and 7.227 ms for (4, 4).

The median and P95 measurements also reveal the index-dependent burstiness of BDS traversal. Consequently, the mean alone does not fully characterize signing behavior. For example, the (6, 14, 16, 16) configuration has a mean of 8.482 ms, a median of 7.064 ms, and a P95 of 13.266 ms. These differences motivate retaining median and tail statistics alongside mean latency.

4.2 Exact Artifact-Size Structure

The measured signature and serialized-state sizes follow the analytical models exactly. For fixed total height $H = 20$, signature size depends on the ordered WOTS parameters but not on how the total tree height is partitioned between the two layers:

$$|\sigma| = 675 + W_0 + W_1.$$

This gives 4963 bytes for (16, 16), 7075 bytes for both (16, 4) and (4, 16), and 9187 bytes for (4, 4).

The equality of the 7075-byte signatures for (16, 4) and (4, 16) is particularly useful for separating communication cost from computation. Although the two placements transmit identical signature sizes, their signing behavior differs substantially because the lower-layer WOTS parameter determines work performed for every signature.

Serialized state follows

$$|\text{SK}| = 2654 + 120h_0 + W_1.$$

Thus, increasing h_0 by one increases serialized state by exactly 120 bytes for fixed w_1 . Changing the upper-layer parameter from $w_1 = 16$ to $w_1 = 4$ increases state by exactly 2112 bytes, corresponding to the difference between the cached upper-layer WOTS-signature sizes.

Across all 20 configurations, both analytical formulas reproduce the measured sizes with zero-byte residuals. The serialized-state quantity includes traversal state and cached upper-layer WOTS-signature material; it is therefore an implementation-state measure rather than a measure of exclusively secret data.

4.3 Rollover Is a Work-Composition Effect

Figure 2 reports the latency change at the lower-tree rollover boundary relative to the mean of the immediately preceding and following signatures. The results show that rollover is not a fixed positive surcharge.

For $(w_0, w_1) = (4, 16)$, rollover latency increases by 64.13–74.29% across the tested height allocations. By contrast, (16, 4) produces negative rollover changes for every height allocation, ranging from -10.94% to -6.57% . The (16, 16) configurations show positive changes of 18.33–24.04%, while (4, 4) produces smaller positive changes of 13.57–17.83%.

The hardware-cycle and logical-work measurements follow the same qualitative pattern. Averaged across height allocations, the (4, 16) placement increases rollover latency by approximately 70.08%, hardware cycles by 70.09%, and logical hash work by 71.28%. For (16, 4), the corresponding averages are approximately -8.53% , -8.53% , and -8.92% , respectively. This agreement indicates that the placement-dependent rollover effect is primarily structural rather than an artifact of wall-clock noise.

The counter decomposition further shows that rollover changes the mix of lower-layer traversal work and upper-layer WOTS work. Because the boundary operation both changes BDS state and generates a fresh upper-layer WOTS signature, its cost cannot be represented accurately as a single constant additive penalty.

4.4 Aggregate Workload Model

The primitive instrumentation counters are strongly collinear. In particular, WOTS chain steps are identical to F -hash calls in the instrumented implementation, while PRF, F -hash, and H -hash counts also exhibit substantial dependence. Primitive-level regression coefficients are therefore not interpreted as independent operation costs.

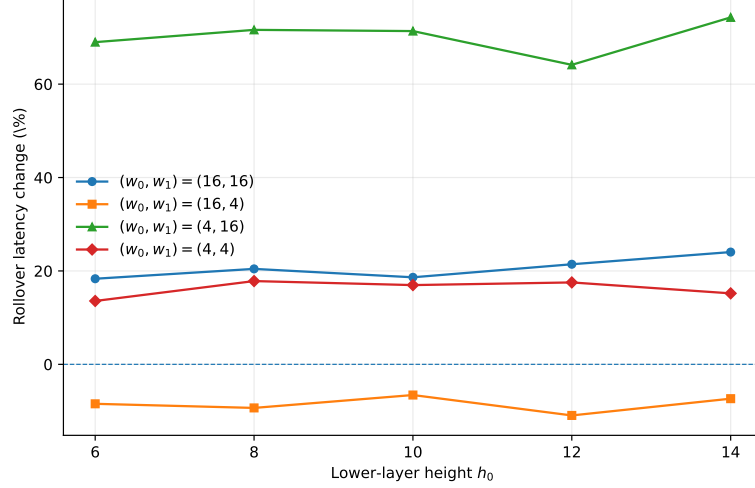


Fig. 2 Rollover latency change relative to the mean of the immediately preceding and following signatures. Negative values indicate that the rollover signature is faster than its local adjacent-signature baseline.

Instead, the aggregate model

$$C = \alpha + \beta N_{\text{logical}} + \gamma N_{\text{tree}}$$

is fitted over all 87,316 signing observations. The resulting model is

$$C = -157535.67 + 1779.238 N_{\text{logical}} + 56185.960 N_{\text{tree}}.$$

The fit achieves

$$R^2 = 0.9880,$$

with a median absolute percentage error of 4.348% and a 95th-percentile absolute percentage error of 6.757%. Figure 3 compares measured and predicted CPU cycles.

These coefficients should be interpreted as an empirical calibration of this Raspberry Pi 5 implementation and workload. In particular, the coefficient associated with logical hash invocations is not a measurement of the universal cost of one SHA-256 compression or one cryptographic hash primitive.

4.5 Multi-Objective Deployment Trade-offs

The primary Pareto analysis jointly minimizes mean signing latency, P95 signing latency, signature size, and serialized state size. Three configurations lie on the strict four-objective front: (6, 14, 16, 16), (6, 14, 4, 16), and (6, 14, 4, 4).

These configurations represent different trade-offs rather than a single universally preferable parameterization. The (6, 14, 16, 16) configuration has the smallest signature among the three at 4963 bytes, with a mean signing latency of 8.482 ms and serialized state of 5518 bytes. The (6, 14, 4, 16) configuration reduces mean signing latency to

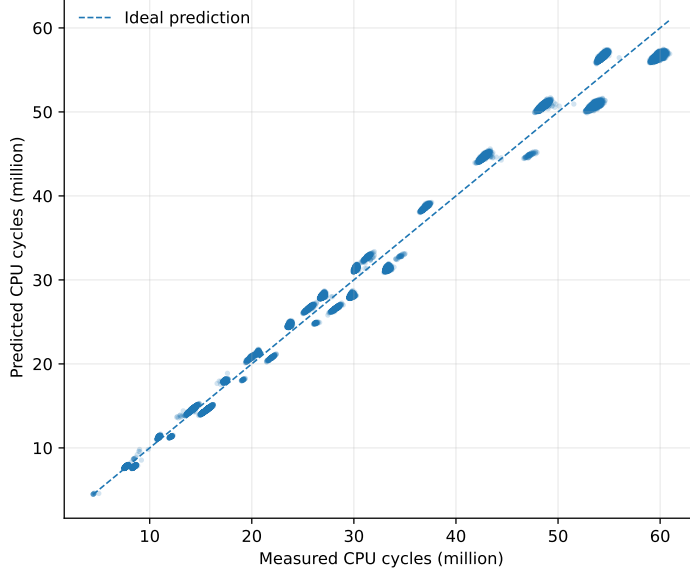


Fig. 3 Measured versus predicted signing CPU cycles for the aggregate workload model over all 87,316 signing observations. The dashed line denotes ideal prediction.

4.623 ms while retaining the same 5518-byte serialized state, at the cost of increasing the signature to 7075 bytes. The (6, 14, 4, 4) configuration has a slightly lower mean signing latency of 4.599 ms and a mean verification latency of 1.608 ms, but increases the signature to 9187 bytes and serialized state to 7630 bytes.

Tolerance analysis shows that the strict front is partly sensitive to small differences. Under symmetric 2%, 5%, and 10% ϵ -dominance, (6, 14, 16, 16) and (6, 14, 4, 16) remain on the front at every tested tolerance, whereas (6, 14, 4, 4) is retained only under strict dominance. Table 2 reports this sensitivity.

Figure 4 visualizes the relation between signing latency, signature size, serialized state, and strict Pareto membership. The results reinforce that WOTS placement cannot be selected from signature size alone: (16, 4) and (4, 16) both produce 7075-byte signatures, yet their signing and rollover costs differ markedly.

A secondary analysis includes mean verification latency as a fifth objective. The strict front expands because small verification differences prevent several configurations from dominating one another. However, under 5% and 10% tolerance the verification-inclusive front contracts to the same three (6, 14) configurations identified above. This sensitivity indicates that conclusions based solely on strict dominance should be interpreted cautiously when objective differences are small.

4.6 Timing Outlier and Robustness of Interpretation

The largest observed wall-clock signing latency is 45.239 ms for $(h_0, h_1, w_0, w_1) = (12, 8, 16, 16)$. This observation is retained in the dataset. It occurs within a short cluster of elevated wall-clock measurements around the same signature indices.

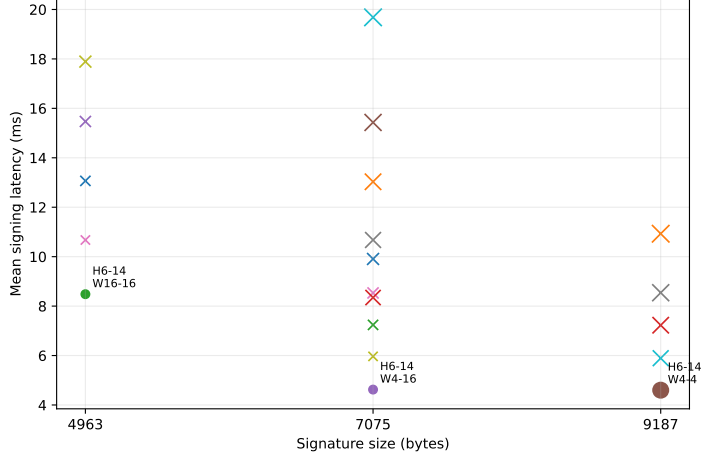


Fig. 4 Deployment trade-off between mean signing latency and signature size. Marker area reflects serialized-state size; strict Pareto configurations are labelled.

The corresponding cycle increase is substantially smaller than the wall-clock increase: the maximum latency is approximately 2.54 times the local median, whereas the hardware-cycle ratio is approximately 1.17. During the campaign the measured temperature remained within the observed operating range and the throttling indicator remained 0x0. The available measurements are therefore consistent with a host-level scheduling, interrupt, or operating-system disturbance, although the experiment does not record sufficient information to identify a specific cause.

For this reason, the observation is not deleted. Median, P95, hardware cycles, and logical-work counters are reported alongside mean and maximum latency so that deployment conclusions do not depend on a single wall-clock extreme.

5 Discussion

5.1 Implications of Ordered Layer Parameterization

The results show that heterogeneous XMSSMT parameterization cannot be characterized solely by the unordered set of layer parameters. Placement matters because the two layers perform different roles during normal signing and at lower-tree rollover boundaries.

The strongest example is the comparison between $(w_0, w_1) = (4, 16)$ and $(16, 4)$. Both placements produce the same 7075-byte signature because the transmitted signature contains one WOTS signature from each layer and therefore depends on the sum $W_0 + W_1$. Their execution costs, however, differ substantially. The lower-layer WOTS signature is generated for every signing operation, whereas the upper-layer WOTS signature is cached and refreshed only at a lower-tree rollover. Consequently, placing $w = 4$ at the lower layer reduces routine signing cost, while placing it only at the upper layer does not provide the same full-cycle benefit.

This distinction is deployment-relevant because two configurations with the same communication cost can have substantially different signing latency, processor-cycle demand, serialized state, and rollover behavior. Signature size alone is therefore insufficient for selecting an ordered heterogeneous XMSSMT configuration.

5.2 Why Lower-Layer Height Dominates Full-Cycle Cost

Increasing h_0 raises the amount of lower-layer traversal work and lengthens the interval between upper-layer rollover events. In the evaluated FAST/BDS implementation with $k = 0$, the initial TreeHash update budget is

$$U_0 = \frac{h_0}{2}.$$

The serialized traversal topology also stores the lower-layer BDS state twice—as current and next state—but the upper-layer state only once. This asymmetry explains why lower-layer height affects both computation and serialized-state size more strongly than an equivalent increase in upper-layer height.

For fixed total height $H = 20$, increasing h_0 by one increases the derived serialized state by exactly 120 bytes. This relation is not an empirical regression: it follows directly from the implemented BDS serialization structure for $n = 32$ and $k = 0$ and is reproduced exactly by all 20 measured configurations.

The performance trend is similarly consistent across the parameter matrix. Moving from $(h_0, h_1) = (6, 14)$ toward $(14, 6)$ generally increases mean signing latency and hardware cycles. Thus, for this implementation and workload, allocating a larger fraction of the fixed total height to the lower layer trades less frequent rollover for higher per-signature traversal cost and larger serialized state.

5.3 Rollover Cost Is Not a Constant Penalty

A key consequence of the full-cycle protocol is that rollover behavior can be observed directly rather than inferred from early signature indices. The measured rollover change is strongly dependent on ordered WOTS placement and is not uniformly positive.

For $(4, 16)$, the rollover signature is substantially more expensive than its adjacent-signature baseline. For $(16, 4)$, the rollover signature is consistently faster than the local baseline across all five tested height allocations. The corresponding hardware-cycle and logical-work changes closely track the wall-clock changes.

This behavior follows from the work composition at the boundary. Rollover does not simply add one upper-layer WOTS computation to an otherwise identical signing operation. It also changes the BDS state, swaps the precomputed next lower-layer state into the current position, reinitializes traversal work, and changes the mixture of lower- and upper-layer operations. Accordingly, the rollover effect should be described as a boundary-specific composition of work rather than as a fixed surcharge.

This interpretation also explains why immediate-neighbor comparison is more informative than quoting rollover latency in isolation. The local baseline partially controls for the index-dependent burst structure of BDS traversal.

5.4 Relationship Between Logical Work and Processor Cost

The aggregate workload model provides a compact explanation for most of the measured cycle variation. Over 87,316 signing observations, logical hash work and generated tree nodes explain approximately 98.8% of the variance in measured CPU cycles.

The model is useful as an implementation-level validation tool rather than as a primitive benchmark. The logical counters intentionally represent XMSS/XMSSMT algorithmic operations. They do not count SHA-256 compression calls, and several primitive counters are strongly collinear. In particular, WOTS chain steps and F -hash calls are identical in the instrumented code. Therefore, interpreting separate primitive regression coefficients as independent cryptographic operation costs would be misleading.

The aggregate model avoids that interpretation and instead establishes that the measured performance differences are closely associated with changes in algorithmic work. The remaining prediction error includes implementation overhead, instruction-level effects, memory behavior, operating-system activity, and other costs not represented by the two aggregate predictors.

5.5 Deployment Trade-offs

The Pareto analysis illustrates that heterogeneous parameter selection is inherently multi-objective. No single configuration simultaneously minimizes signing latency, verification latency, signature size, and serialized state.

The strict four-objective front contains three configurations from the $(h_0, h_1) = (6, 14)$ family. The $(16, 16)$ WOTS placement provides the smallest signature among them, whereas $(4, 16)$ substantially reduces signing latency while preserving the same serialized-state size. The $(4, 4)$ placement slightly reduces mean signing latency further but requires both a larger signature and larger serialized state.

The tolerance analysis is important because strict Pareto membership can be unstable when objective differences are small. The $(6, 14, 4, 4)$ configuration disappears from the front under 2%, 5%, and 10% symmetric tolerance, whereas $(6, 14, 16, 16)$ and $(6, 14, 4, 16)$ remain. This indicates that the latter two configurations represent more robust trade-off points within the measured objective set.

The verification-inclusive analysis further demonstrates the sensitivity of strict dominance to additional objectives. Adding mean verification latency expands the strict front, but a 5% or 10% tolerance again reduces the front to the same three $(6, 14)$ configurations. These results support reporting both strict and tolerance-based Pareto analyses rather than drawing deployment conclusions from a single strict front.

5.6 Contribution Relative to Prior XMSSMT Parameter Studies

The contribution of this study is not the claim that layer-specific XMSSMT parameters are themselves new. Earlier XMSSMT parameter work has considered layer-dependent

quantities, including subtree heights, traversal parameters, and Winternitz parameters. The present study instead focuses on an implementation and measurement gap.

Specifically, the study combines layer-specific height and WOTS parameterization with a FAST/BDS XMSSMT implementation; evaluates ordered placement effects under one matched full-cycle protocol; measures both wall-clock latency and hardware CPU cycles at operation level; records algorithmic work counters; validates serialized-state and signature-size models exactly; measures lower-tree rollover behavior directly; and evaluates deployment trade-offs using strict and tolerance-based Pareto analysis.

This framing is important because several of the observed effects are structural consequences of XMSSMT and WOTS organization rather than new cryptographic mechanisms. The empirical contribution is the quantification and validation of those effects under a controlled, matched Raspberry Pi 5 implementation.

5.7 Limitations

Several limitations constrain the scope of the conclusions.

First, the evaluated configurations are experimental heterogeneous XMSSMT configurations. They are not standardized XMSSMT parameter sets, and no standardized OIDs are claimed for them. The results therefore characterize implementation and deployment trade-offs rather than proposing direct replacements for standardized parameter sets.

Second, the study fixes $H = 20$, $d = 2$, $n = 32$, and BDS $k = 0$. Other total heights, numbers of layers, digest sizes, or traversal parameters may change both the absolute costs and the relative importance of layer placement. In particular, increasing d introduces additional upper-layer states and rollover levels that are not represented by the two-layer experiment.

Third, only $w \in \{4, 16\}$ is used in the final 20-cell heterogeneous matrix. These values are sufficient to create clear placement contrasts, but they do not exhaust the possible Winternitz parameter space.

Fourth, measurements were obtained on one Raspberry Pi 5 platform using one compiler and one software stack. The implementation invokes OpenSSL SHA-256/SHA-512 through `libcrypto.so.3`, and the CPU exposes the AArch64 `sha2` feature, but the experiment does not trace the exact OpenSSL internal implementation path selected for each hash invocation. Consequently, the empirical cycle-model coefficients should not be generalized to other processors or cryptographic libraries.

Fifth, BDS signing costs are index-dependent and serially structured. The observations in a full traversal cycle are therefore not independent and identically distributed samples. The study reports descriptive statistics, hardware cycles, and logical work rather than applying inferential procedures that assume independent signing observations.

Sixth, one configuration contains a pronounced wall-clock timing outlier. The observation is retained because the experiment provides no evidence of invalid cryptographic execution or thermal throttling. Hardware cycles and logical work indicate

that its wall-clock magnitude is not proportional to algorithmic work, making host-level scheduling or interrupt activity a plausible explanation. However, the precise cause was not instrumented and therefore cannot be established.

Finally, the Pareto results depend on the selected objective set. Alternative deployment contexts may assign different importance to key-generation latency, verification throughput, energy, RAM usage, flash footprint, state-write durability, or communication bandwidth. The reported Pareto sets should therefore be interpreted as objective-dependent trade-off surfaces rather than universal parameter recommendations.

5.8 Summary of Design Implications

Within the evaluated design space, four implementation-level conclusions are supported consistently by the analytical and empirical results.

First, lower-layer WOTS placement governs routine signing cost more strongly than upper-layer WOTS placement because the lower-layer WOTS signature is regenerated on every signature.

Second, lower-layer height controls both traversal work and serialized state through the asymmetric FAST/BDS state topology.

Third, rollover behavior depends on the composition of work at the boundary and can be either more expensive or less expensive than nearby signatures.

Fourth, communication cost, state cost, and signing cost cannot be inferred from one another. Parameter selection therefore requires joint consideration of ordered layer structure and deployment objectives.

6 Conclusion

This study evaluated ordered layer-specific tree-height and WOTS parameterization in a two-layer XMSSMT FAST/BDS implementation using a matched full-cycle Raspberry Pi 5 benchmark. The revised protocol measures one complete lower-layer traversal cycle plus rollover for each of 20 experimental configurations, producing 87,316 signing observations together with matched verification measurements, hardware-cycle counts, and logical-work counters.

The results show that ordered placement matters. Lower-layer WOTS selection dominates routine signing cost, whereas upper-layer WOTS selection has a stronger influence on rollover behavior and serialized state. Configurations with $(w_0, w_1) = (4, 16)$ and $(16, 4)$ produce the same 7075-byte signature, yet their computational and state behavior differ substantially because the two layers perform different roles.

Tree-height allocation also produces a systematic structural effect. For fixed total height $H = 20$, increasing the lower-layer height raises serialized state by 120 bytes per level under the measured FAST/BDS topology, while full-cycle signing cost increases because additional lower-layer traversal work is exercised more frequently.

The analytical artifact-size model reproduces all measured signature and serialized-state sizes exactly. The aggregate workload model also explains most measured CPU-cycle variation, with $R^2 = 0.9880$, while remaining explicitly an empirical platform/workload calibration rather than a primitive hash-cost model.

Rollover measurements further show that subtree transition cost is not a constant positive surcharge. Its effect depends on the ordered WOTS placement and on how routine traversal work and upper-layer WOTS work are composed at the boundary.

Under the primary four-objective analysis of mean signing latency, P95 signing latency, signature size, and serialized state, three configurations lie on the strict Pareto front. Two of these, (6, 14, 16, 16) and (6, 14, 4, 16), remain on the front under 2%, 5%, and 10% symmetric tolerance, whereas (6, 14, 4, 4) is retained only under strict dominance. These results reinforce that XMSSMT parameter selection is deployment-dependent and should not be reduced to a single universal optimum.

The contribution is therefore not a new XMSSMT primitive or the first use of heterogeneous parameters. It is a matched implementation-level quantification of ordered layer effects, traversal behavior, rollover composition, serialized-state structure, and multi-objective deployment trade-offs. Future work should extend the analysis to additional total heights, layer counts, Winternitz parameters, BDS settings, processor classes, and standardized deployment contexts.

Acknowledgements. The authors acknowledge the support of the School of Computer Sciences, Universiti Sains Malaysia, and the Cybersecurity Research Centre (CYRES).

Declarations

Author contributions. Sohaib Rana: Conceptualization, methodology, software, validation, formal analysis, investigation, data curation, visualization, and writing – original draft.

Mohd Najwadi Yusoff: Supervision, conceptualization, methodology, project administration, and writing – review and editing.

Je Sen Teh: Supervision, methodology, validation, and writing – review and editing.

Adnan Anwar: Validation and writing – review and editing.

Funding. The authors received no specific funding for this work.

Data availability. The processed benchmark datasets, normalized operation-level measurements, analysis outputs, provenance records, and integrity manifests supporting this study are provided in Online Resource 1 and in the public reproducibility repository at <https://github.com/sohaibrana1/xmssmt-layer-adaptive-rpi5>.

Code availability. The validated heterogeneous XMSSMT source modifications, analysis scripts, provenance records, and reproducibility artifacts supporting this study are available in the public repository at <https://github.com/sohaibrana1/xmssmt-layer-adaptive-rpi5>.

Competing interests. The authors declare that they have no competing interests.

References

- [1] Buchmann, J., Dahmen, E., Hülsing, A.: XMSS: A practical forward secure signature scheme based on minimal security assumptions. In: Post-Quantum Cryptography. Lecture Notes in Computer Science, vol. 7071, pp. 117–129. Springer, ??? (2011). https://doi.org/10.1007/978-3-642-25405-5_8
- [2] Hülsing, A., Butin, D., Gazdag, S.-L., Rijneveld, J., Mohaisen, A.: XMSS: extended merkle signature scheme. RFC 8391, Internet Research Task Force (May 2018). <https://doi.org/10.17487/RFC8391>
- [3] Hülsing, A.: W-OTS+: Shorter signatures for hash-based signature schemes. In: Progress in Cryptology – AFRICACRYPT 2013. Lecture Notes in Computer Science, vol. 7918, pp. 173–188. Springer, ??? (2013). https://doi.org/10.1007/978-3-642-38553-7_10
- [4] McGrew, D., Curcio, M., Fluhrer, S.: Leighton-micali hash-based signatures. RFC 8554, Internet Research Task Force (April 2019). <https://doi.org/10.17487/RFC8554>
- [5] Cooper, D.A., Apon, D.C., Dang, Q.H., Davidson, M.S., Dworkin, M.J., Miller, C.A.: Recommendation for stateful hash-based signature schemes. NIST Special Publication 800-208, National Institute of Standards and Technology (October 2020). <https://doi.org/10.6028/NIST.SP.800-208>
- [6] Hülsing, A., Rausch, L., Buchmann, J.: Optimal parameters for XMSSMT. In: Security Engineering and Intelligence Informatics. Lecture Notes in Computer Science, vol. 8128, pp. 194–208. Springer, ??? (2013). https://doi.org/10.1007/978-3-642-40588-4_14